

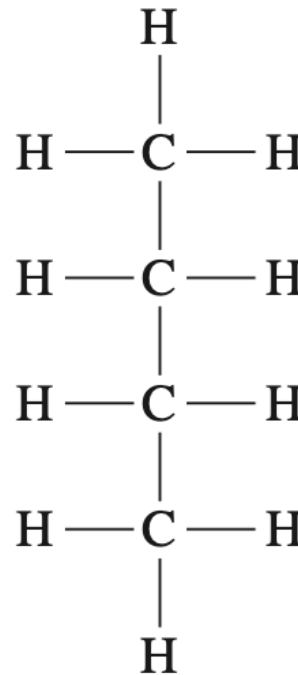
Cây (Tree)

Cây

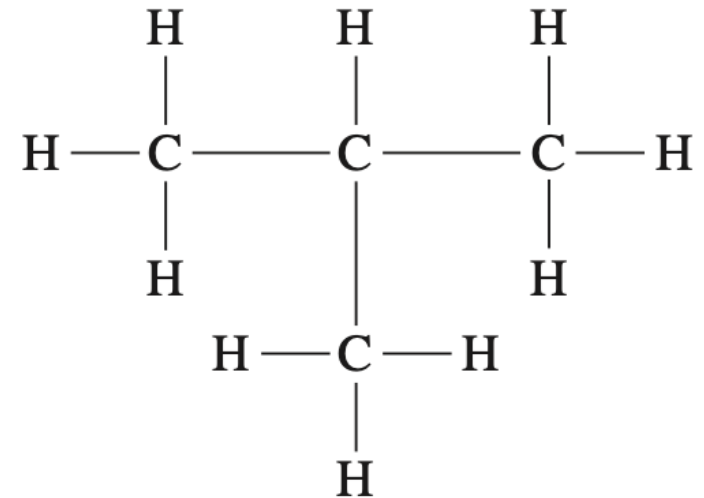
□ Lịch sử ra đời

Được đề xuất bởi
nhà toán học người
Anh Arthur Cayley
vào năm 1857

Để liệt kê các đồng
phân của các hợp
chất có dạng C_nH_{2n+2}



Butane



Isobutane

Cây

- ❑ Cây là đồ thị vô hướng liên thông không có chu trình đơn
- ❑ Rừng (forest)
 - Đồ thị không chứa bất kỳ chu trình đơn nào liên kết với nhau
- ❑ **Định lý:** Một đồ thị vô hướng là cây nếu và chỉ nếu tồn tại một đường đi duy nhất giữa hai đỉnh bất kỳ nào của nó.

Cây có gốc (Rooted tree)

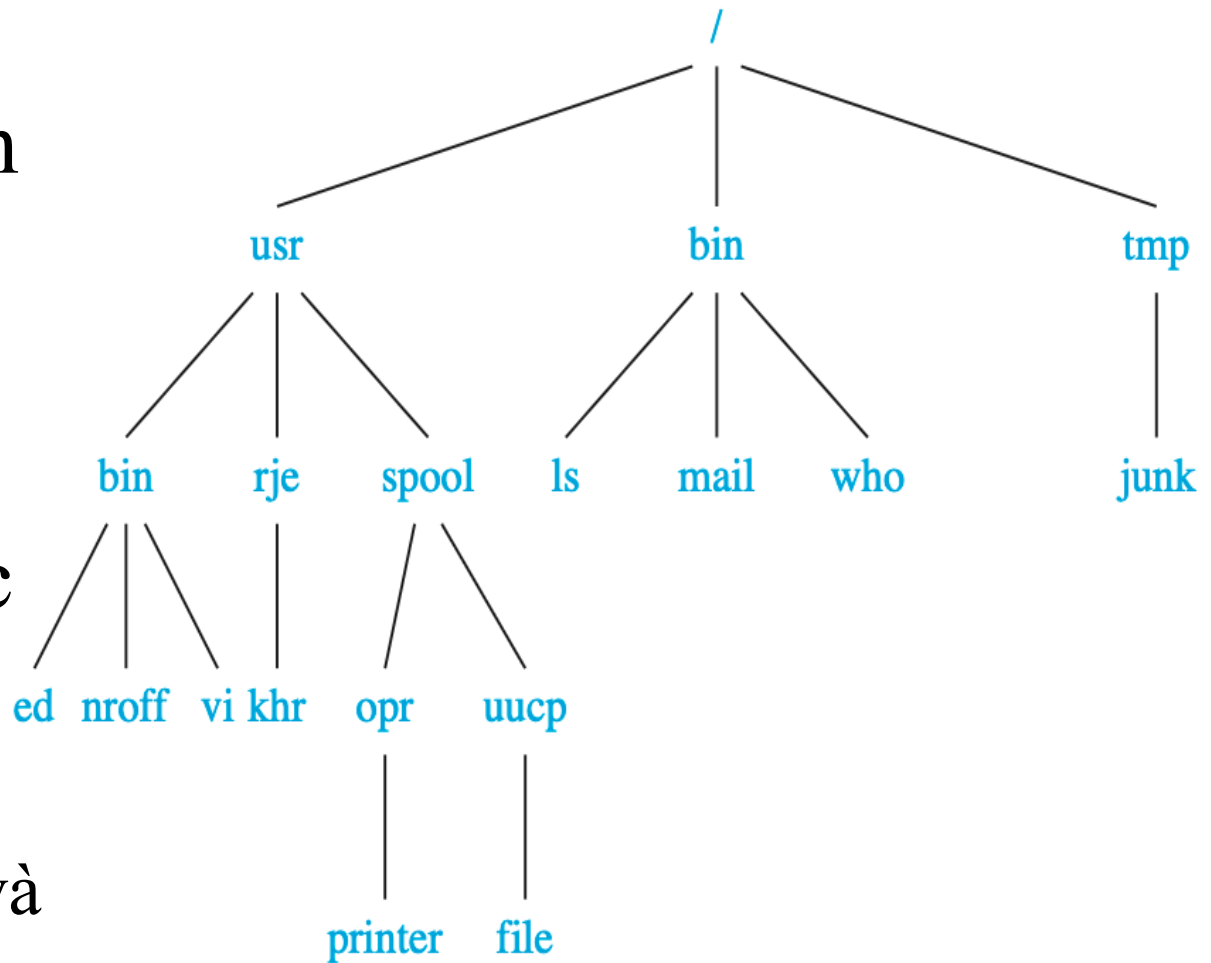
- ❑ Một đỉnh được chỉ định làm gốc và mọi cạnh được hướng ra từ gốc.

Cha mẹ, con, các đỉnh trong (internal nodes), lá

- ❑ Cây (có gốc) được gọi là cây m-ary nếu mỗi đỉnh trong của nó không có nhiều hơn m con.
- ❑ Cây này được gọi là cây m-ary đầy đủ nếu các đỉnh trong của nó có chính xác m con.
- ❑ Với $m = 2$, ta có cây nhị phân.

Cây có gốc sắp xếp

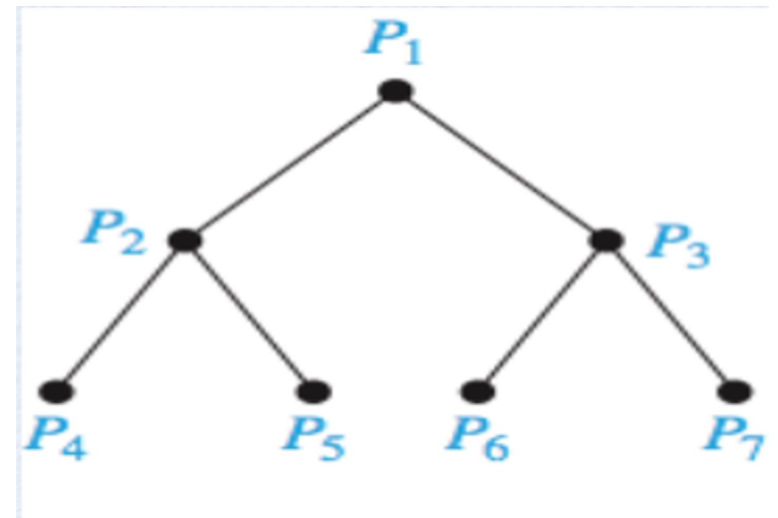
- ❑ Cây (có gốc) trong đó mỗi đỉnh trong được sắp xếp
- ❑ Cây nhị phân sắp xếp (thường được gọi là cây nhị phân)
 - Cây con bên trái và cây con bên phải



Cây

- ❑ Cây như là một mô hình tính toán
 - ❑ Các bộ vi xử lý (CPU) được tổ chức như trong hình 12 được sử dụng nhằm tìm tổng của 8 số.
 - ❑ Quy trình này gồm 3 bước.

Cộng x1 và x2 sử dụng P4
Cộng x3 và x4 sử dụng P5
Cộng x5 và x6 sử dụng P6
Cộng x7 và x8 sử dụng P7



Mạng kết nối dưới dạng cây của 07 bộ vi xử lý.

Các tính chất của cây

- Một cây có n đỉnh có $n-1$ cạnh
 - G liên thông
 - G không có chu trình
 - G có $n - 1$ cạnh

- Khi một trong ba điều kiện trên thỏa mãn, điều kiện còn lại cũng chắc chắn cũng thỏa mãn và G là một cây.

Các tính chất của cây

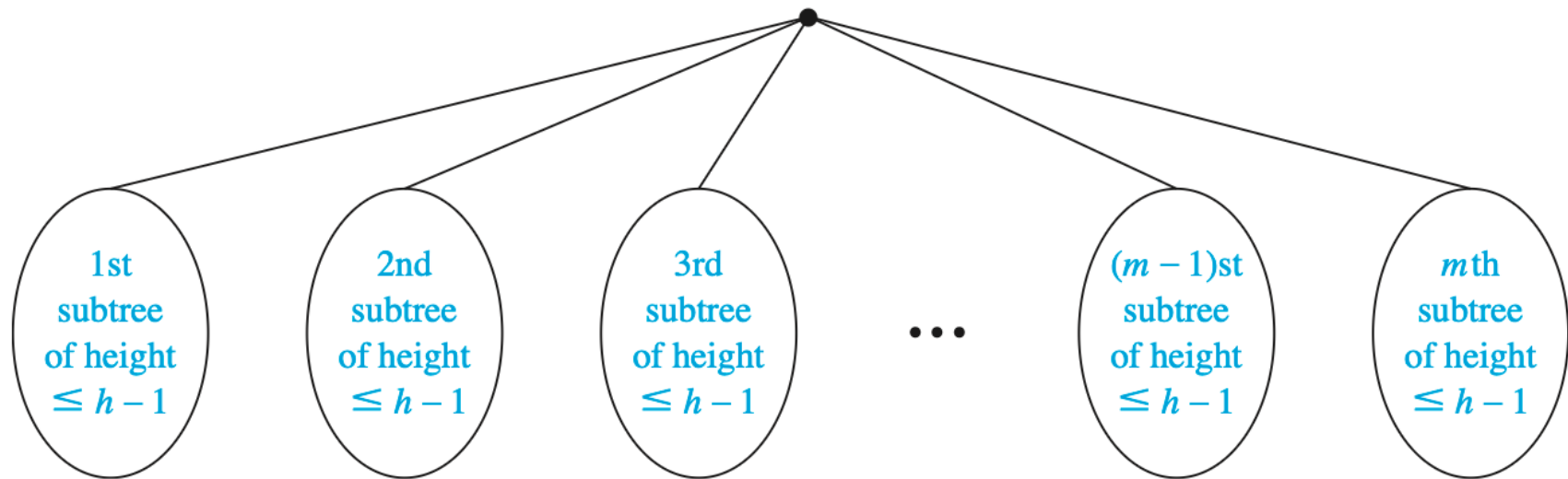
- Một cây đầy đủ m -phân với i đỉnh trong thì chứa $n=mi+1$ đỉnh.
- Một cây đầy đủ m -phân với:
 - n đỉnh thì có $i=(n-1)/m$ đỉnh trong
 - và $l = [(m-1)n + 1]/m$ lá.
 - i đỉnh trong thì có $n=mi+1$ đỉnh và $l=(m-1)i + 1$ lá.
 - l lá thì có $n=(ml-1)/(m-1)$ đỉnh và $i=(l-1)/(m-1)$ đỉnh trong.

Cây *m*-phân đối xứng

- ❑ Bậc của một đỉnh v trong cây có gốc là độ dài của đường đi duy nhất từ gốc tới đỉnh đó.
- ❑ Độ cao của cây có gốc là giá trị lớn nhất của bậc của các đỉnh trong cây đó.

Cây m -phân

- **Định lý:** Tồn tại nhiều nhất mh lá trong một cây m -phân với chiều cao là h .
- Nếu một cây m -phân chiều cao h có l lá thì $h \geq \lceil \log_m(l) \rceil$. Nếu cây đó đầy đủ và đối xứng thì $h = \lceil \log_m(l) \rceil$.



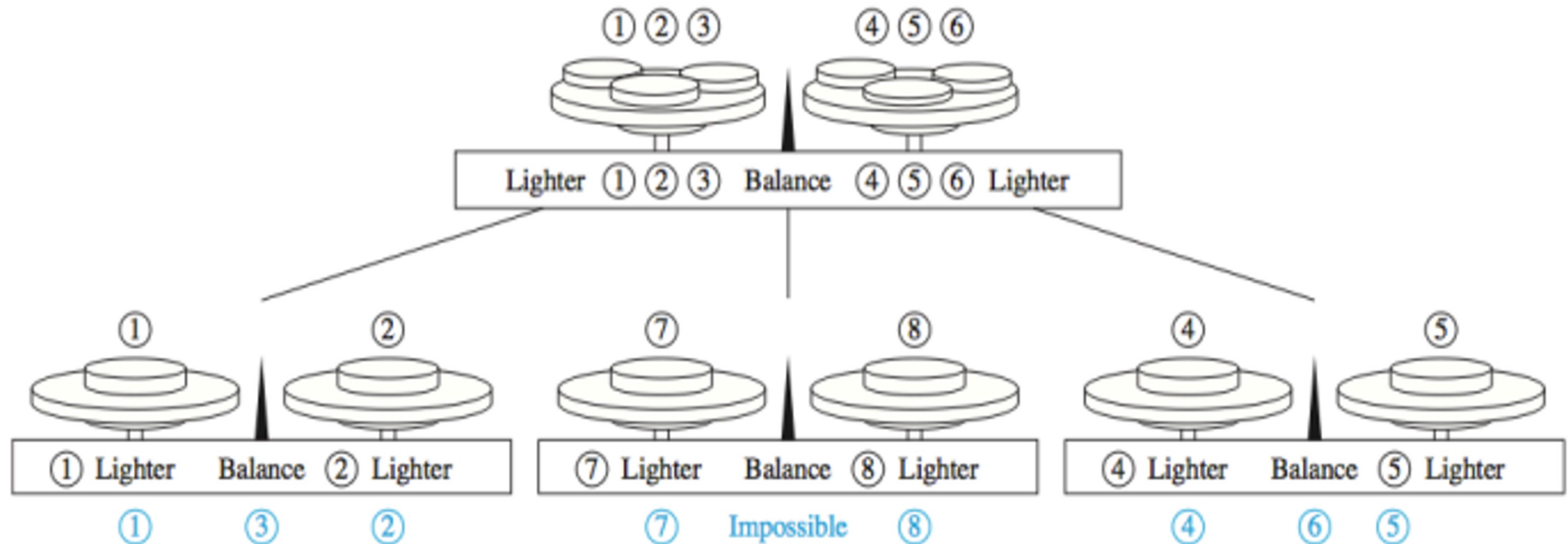
Ứng dụng của cây

- ☐ Lưu trữ các đối tượng của một danh sách
 - ☐ Mỗi đối tượng được tìm thấy một cách dễ dàng?
- ☐ Hỗ trợ ra quyết định
 - ☐ Tìm ra một đối tượng với một tiêu chí cụ thể?
- ☐ Mã hóa một tập các ký tự thành chuỗi bit

Tìm kiếm cây nhị phân

- ❑ Mỗi đỉnh có duy nhất hai con: con trái và con phải.
 - ❑ Mỗi đỉnh chứa một giá trị trong danh sách (mảng) các giá trị.
- ❑ Giá trị của một đỉnh lớn hơn tất cả các giá trị của các đỉnh nằm trong cây con trái và nhỏ hơn tất cả các giá trị của các đỉnh nằm trong cây con phải.

Cây quyết định (decision tree)



Một cây quyết định cho việc phát hiện một đồng xu giả trong 6 đồng xu. (*Lighter*: Nhẹ hơn; *Balance*: Cân bằng; *Impossible*: Không thể xác định). Đồng xu giả được in màu (xanh) ở dưới mỗi lần cân cuối cùng.

Thuật toán sắp xếp

- ❑ **Định lý:** Một thuật toán sắp xếp dựa vào so sánh nhị phân cần ít nhất $[log n!]$ so sánh.
- ❑ **Kết luận:** Số so sánh cần sử dụng trong một thuật toán để sắp xếp n phần tử dựa vào so sánh nhị phân là $\Omega(n \log n)$

Mã tiền tố (prefix code)

- ❑ Các dãy bit có độ dài khác nhau được sử dụng để mã hóa các ký tự.

Các ký tự có tần suất xuất hiện lớn nên được mã hóa bởi dãy bit ngắn

Các dãy bit dài được sử dụng để mã hóa các ký tự ít xuất hiện.

- ❑ Một vài phương pháp được sử dụng nhằm xác định dãy bit cho từng ký tự.

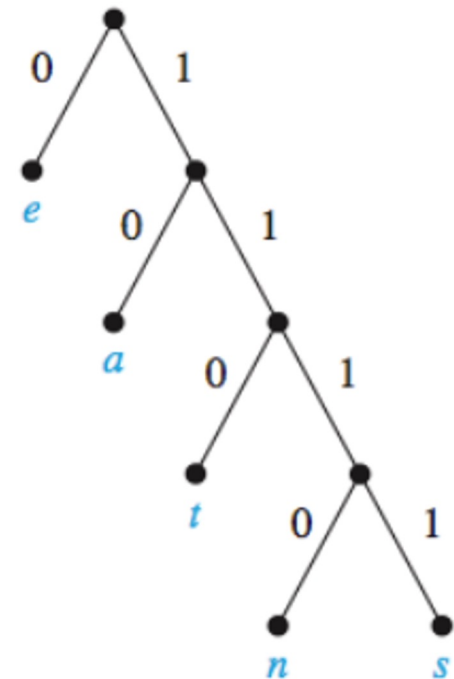
- ❑ Mã tiền tố

Dãy bit mã hóa một ký tự không bao giờ là tiền tố (phần đầu tiên) của dãy bit của các ký tự nào khác.

Mã tiền tố

- ❑ Một mã tiền tố có thể được biểu diễn bằng một cây nhị phân

Các ký tự là nhãn của các lá trên cây
Cạnh nối với con trái được gán là 0 và cạnh được nối với con phải được gán là 1.



Mã hóa Huffman

❑ Được đề xuất bởi David Huffman trong một công trình ông viết vào năm 1951 khi là một nghiên cứu sinh tại MIT.

❑ Thuật toán

Có đầu vào là tần suất xuất hiện của các ký tự trong một chuỗi

Đầu ra là một mã tiền tố nhằm mã hóa chuỗi đó sử dụng ít nhất có thể số lượng bit.

Mã hóa Huffman

Procedure Huffman(C : các ký tự a_i với tần số $w_i, i = 1, \dots, n$)

$F :=$ rừng gồm n cây có gốc, mỗi cây bao gồm chỉ một đỉnh a_i và trọng số w_i

while F không phải là một cây

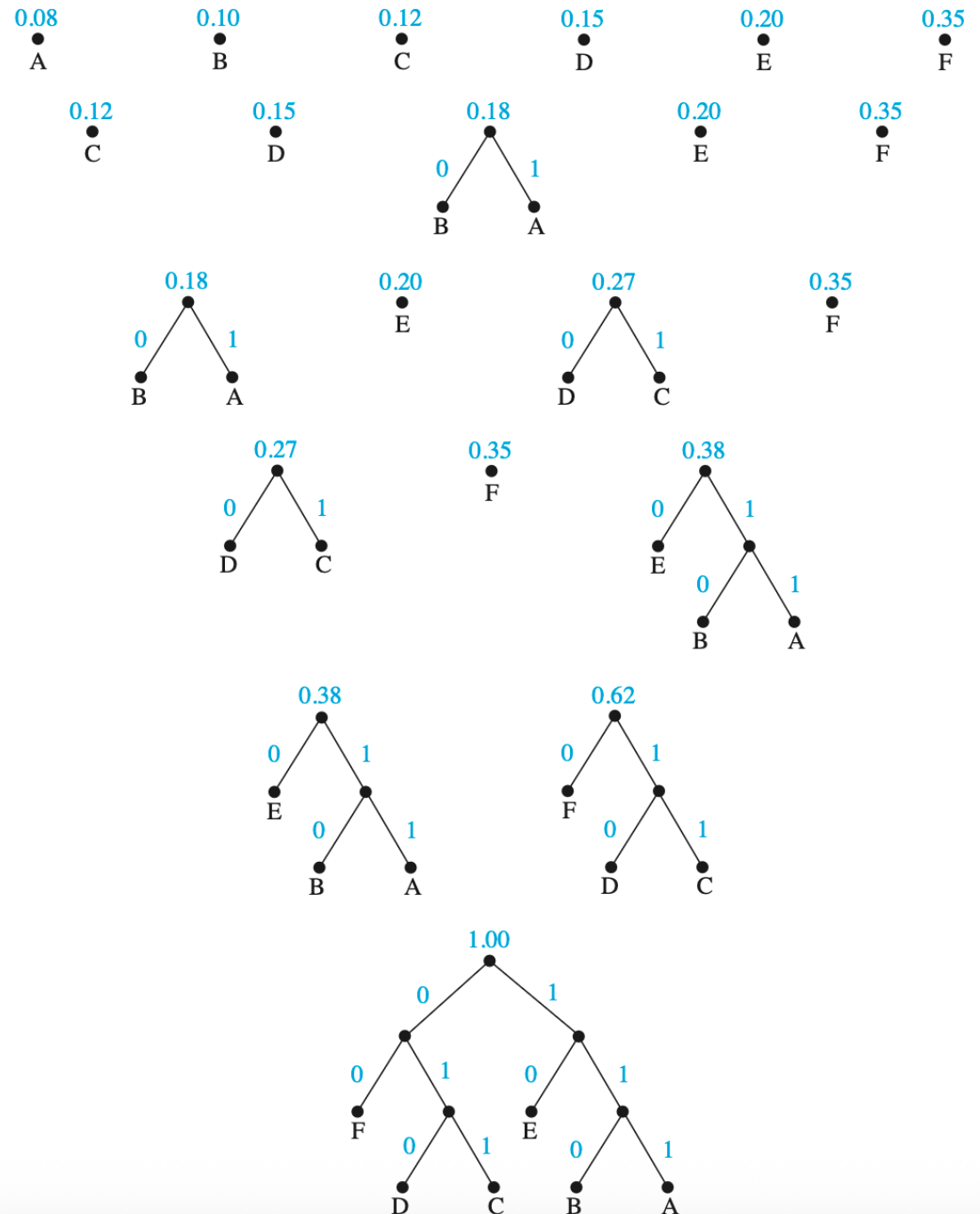
Thay 2 cây T và T' có trọng số nhỏ nhất trong F với $w(T) \geq w(T')$ bằng một cây có gốc mới trong đó T là cây con bên trái còn T' là cây con bên phải. Gán nhãn cạnh mới tới T là 0 còn cạnh mới tới T' là 1.

Gán $w(T) + w(T')$ là trọng số cho cây mới tạo.

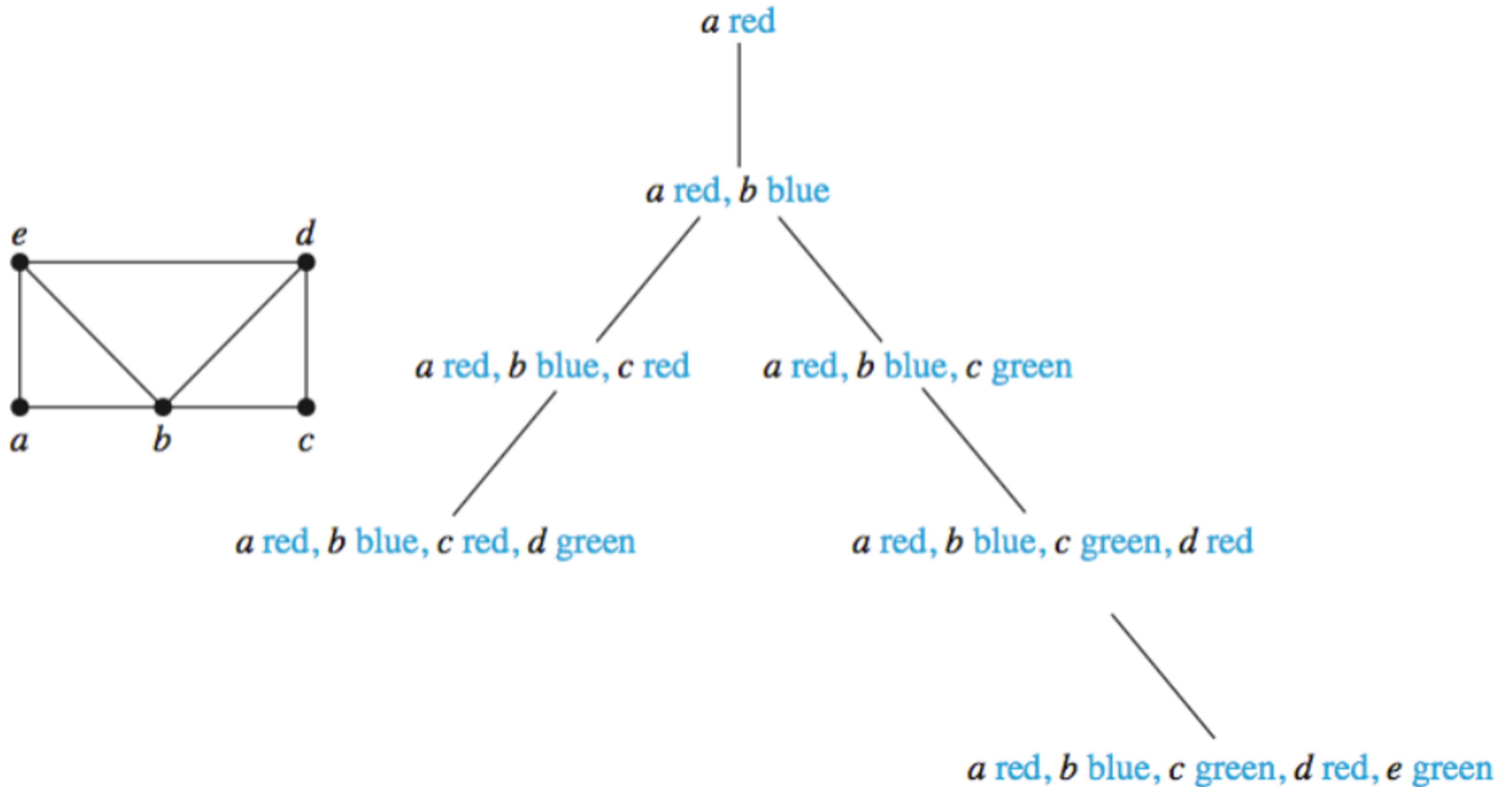
{Mã Huffman cho ký tự a_i là chuỗi các nhãn của các cạnh trên đường đi duy nhất từ gốc tới đỉnh lá a_i }

Mã hóa Huffman

□ A: 0.08, B: 0.10,
C: 0.12, D: 0.15,
E: 0.20, F: 0.35



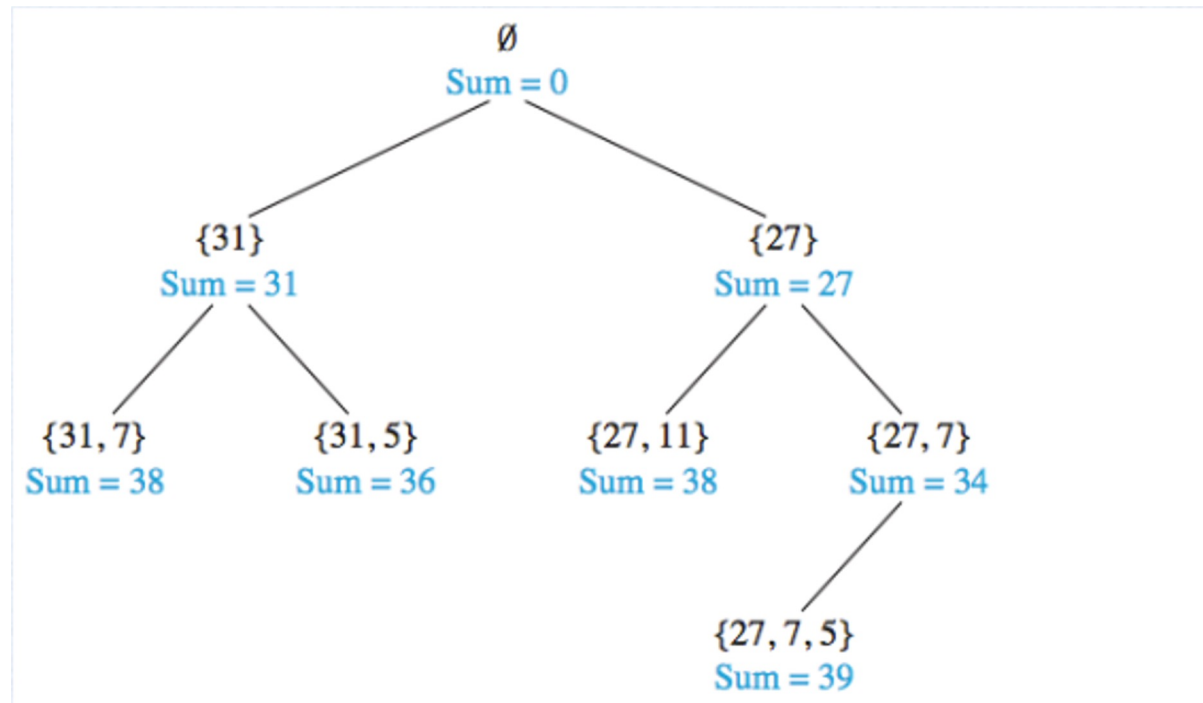
Cây trong thuật toán quay lui



Tô màu đồ thị (bên trái màn hình) sử dụng thuật toán quay lui

Cây trong thuật toán quay lui

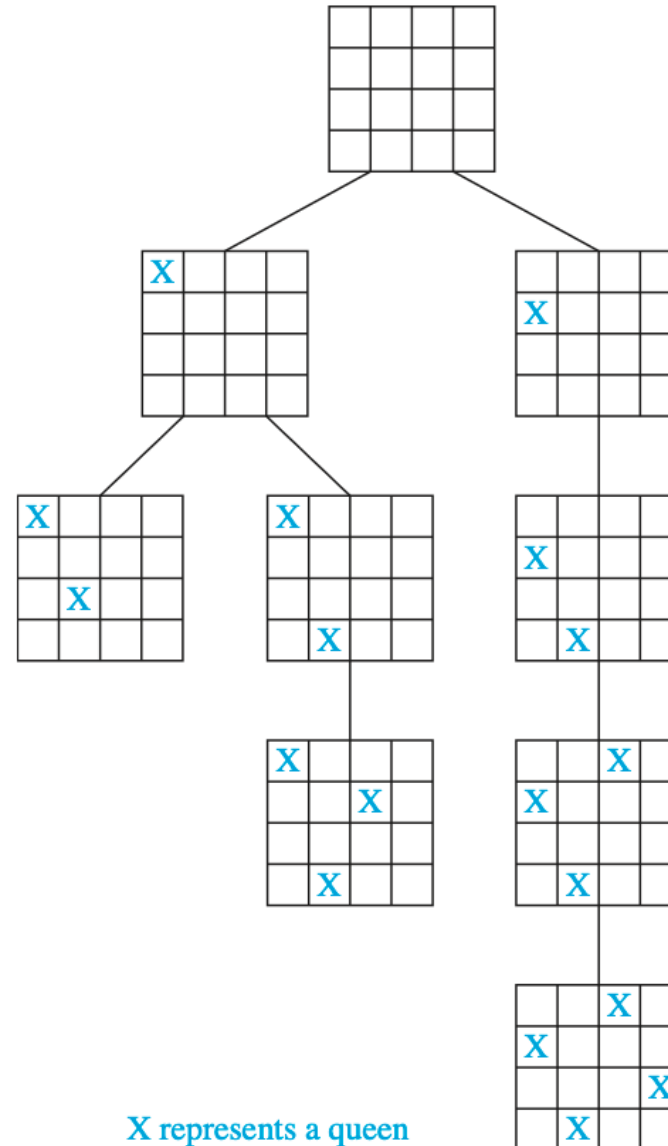
- Tổng của tập con: Cho một tập gồm n phần tử $x_1, x_2, \dots, x_n$. Tìm một tập con mà tổng các phần tử của nó bằng M .



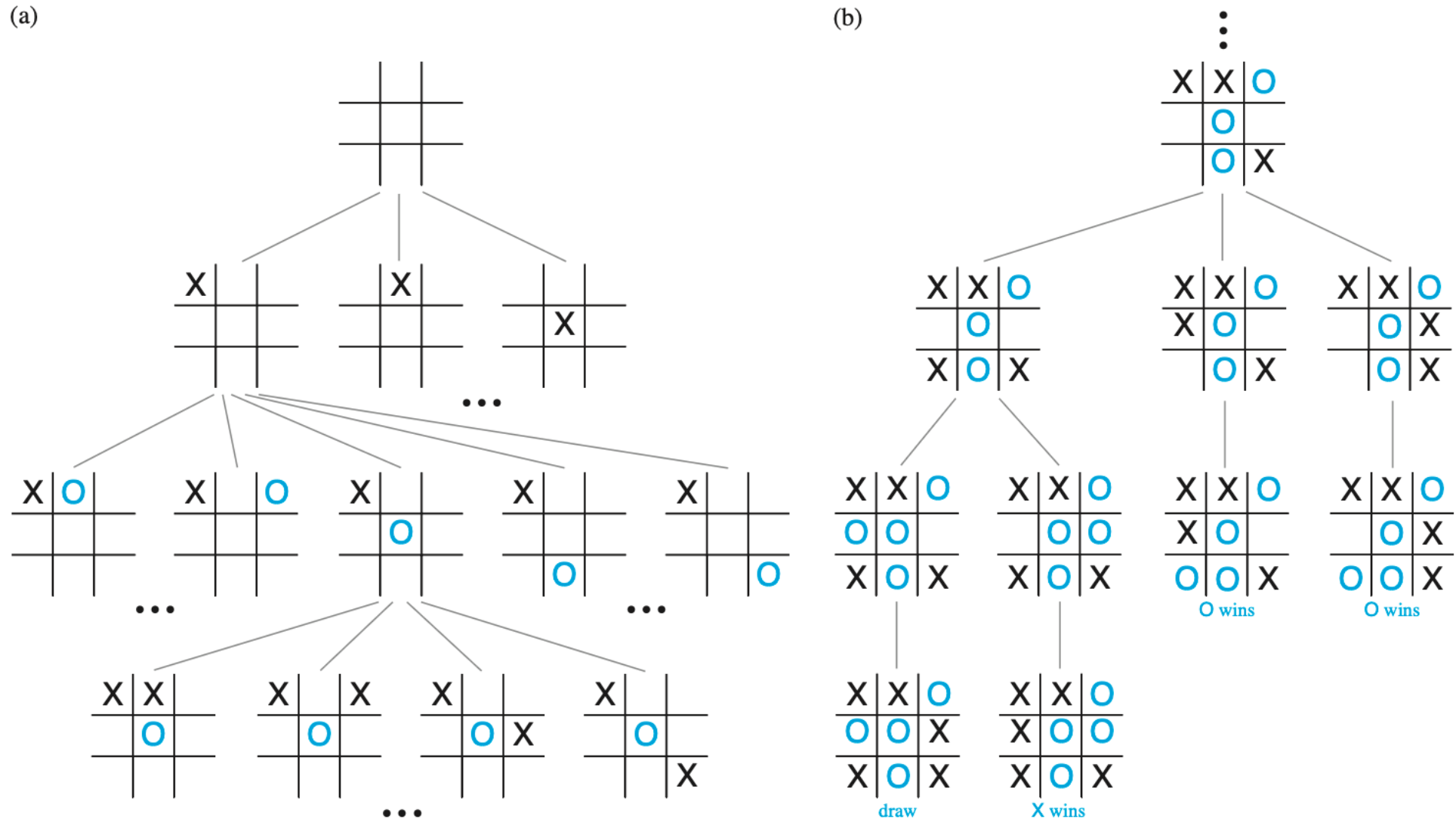
Lược đồ quay lui để tìm các số có tổng bằng 39

Cây trong thuật toán quay lui

□ Bài toán n con hậu

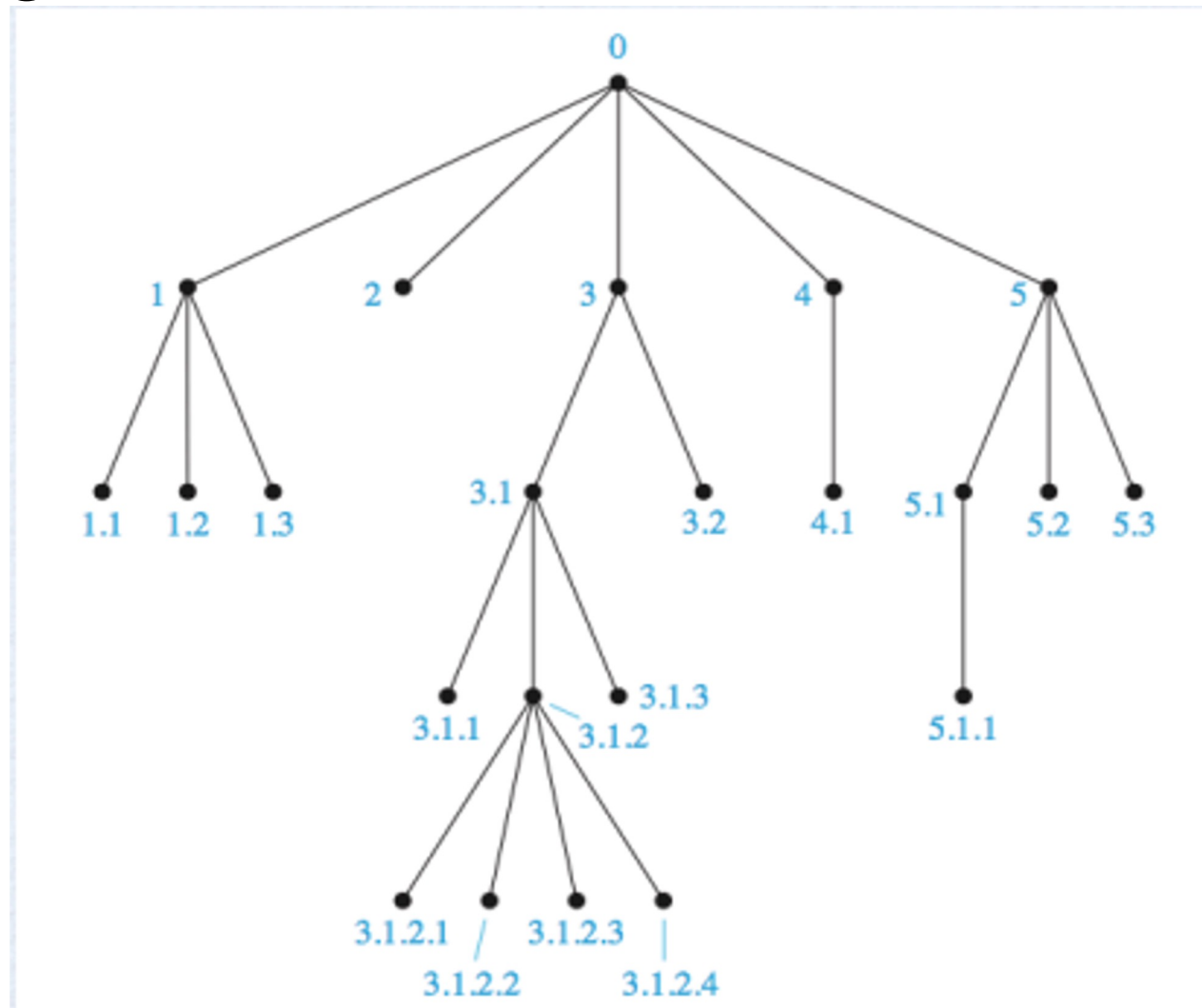


Cây trò chơi



Duyệt cây (tree traversal)

□ Hệ thống đánh địa chỉ



Thuật toán duyệt cây

- ☐ Duyệt tiền tự (pre-order)
- ☐ Duyệt trung tự (in-order)
- ☐ Duyệt hậu tự (post-order)

Duyệt tiền tự

Giả sử T là cây có gốc r đã sắp xếp. Nếu T chứa duy nhất gốc r thì r được duyệt tiền thứ tự. Ngược lại, giả sử $T_1, T_2, \dots, T_n$ là các cây con tại r từ trái sang phải của T . Duyệt tiền thứ tự bắt đầu tại vị trí của r . Nó tiếp tục duyệt tiền thứ tự qua T_1 , rồi đến T_2 , cứ tiếp tục như vậy cho đến khi T_n được duyệt tiền thứ tự

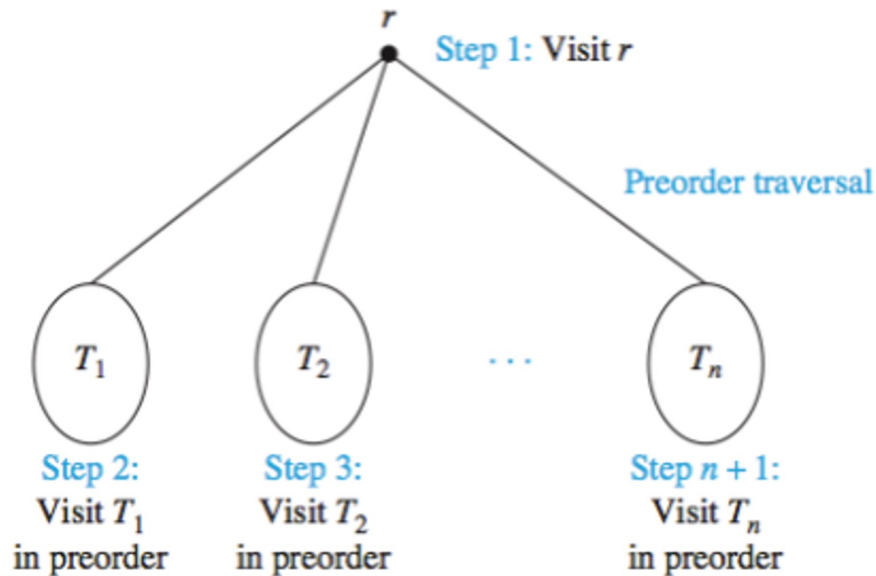


FIGURE 2 Preorder Traversal.

ALGORITHM 1 Preorder Traversal.

```
procedure preorder( $T$ : ordered rooted tree)
 $r := \text{root of } T$ 
list  $r$ 
for each child  $c$  of  $r$  from left to right
     $T(c) := \text{subtree with } c \text{ as its root}$ 
    preorder( $T(c)$ )
```

Duyệt trung tự

Giả sử T là cây có gốc r đã sắp xếp. Nếu T chứa duy nhất gốc r thì r được duyệt trung thứ tự. Ngược lại, giả sử $T_1, T_2, \dots, T_n$ là các cây con tại r từ trái sang phải của T . Duyệt trung thứ tự bắt đầu tại vị trí của T_1 . Tiếp theo, nó duyệt trung thứ tự qua r . Nó tiếp tục duyệt trung thứ tự qua T_2 , rồi đến T_3 , cứ tiếp tục như vậy cho đến khi T_n được duyệt trung thứ tự

ALGORITHM 2 Inorder Traversal.

procedure *inorder*(T : ordered rooted tree)

$r := \text{root of } T$

if r is a leaf **then** list r

else

$l := \text{first child of } r \text{ from left to right}$

$T(l) := \text{subtree with } l \text{ as its root}$

inorder($T(l)$)

list r

for each child c of r **except for** l **from left to right**

$T(c) := \text{subtree with } c \text{ as its root}$

inorder($T(c)$)

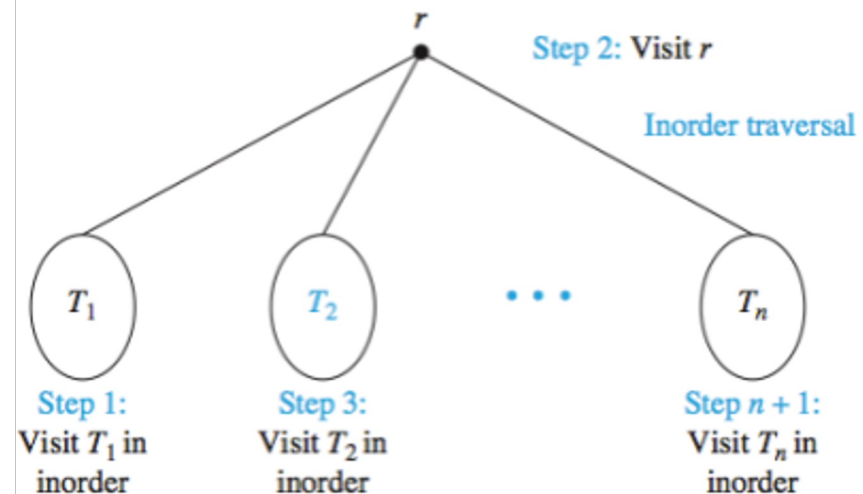
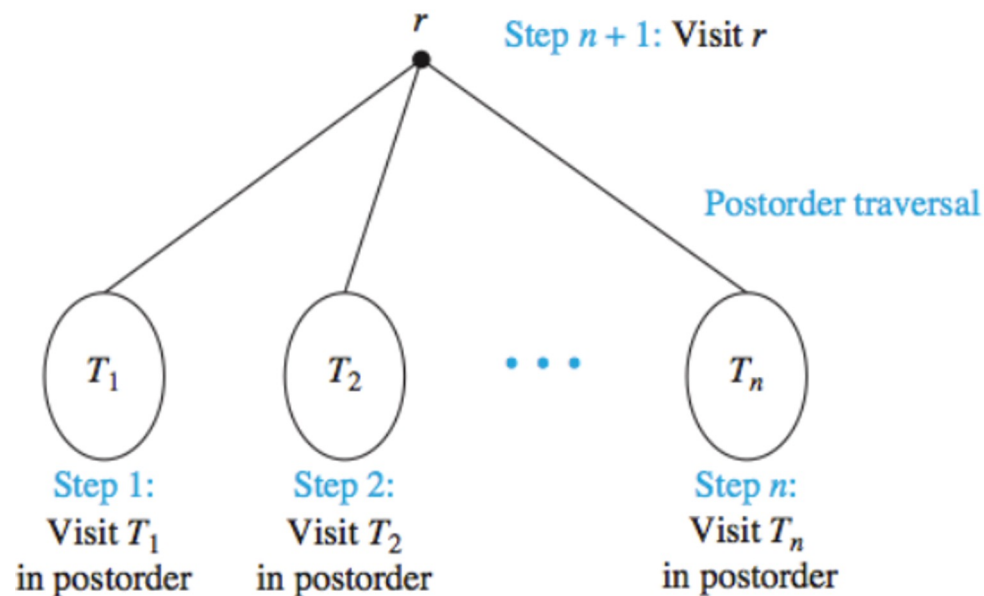


FIGURE 5 Inorder Traversal.

Duyệt hậu tự

Giả sử T là cây có gốc r đã sắp xếp. Nếu T chứa duy nhất gốc r thì r được duyệt hậu thứ tự. Ngược lại, giả sử $T_1, T_2, \dots, T_n$ là các cây con tại r từ trái sang phải của T . Duyệt hậu thứ tự bắt đầu tại vị trí của T_1 . Nó tiếp tục duyệt hậu thứ tự qua T_2 , rồi đến T_3 , cứ tiếp tục như vậy cho đến khi T_n và kết thúc bằng việc duyệt r .



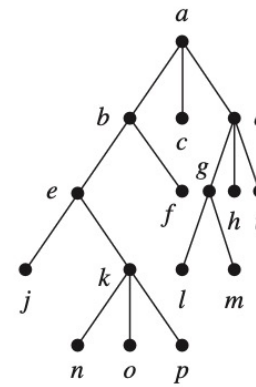
ALGORITHM 3 Postorder Traversal.

```
procedure postorder( $T$ : ordered rooted tree)
 $r := \text{root of } T$ 
for each child  $c$  of  $r$  from left to right
     $T(c) := \text{subtree with } c \text{ as its root}$ 
    postorder( $T(c)$ )
list  $r$ 
```

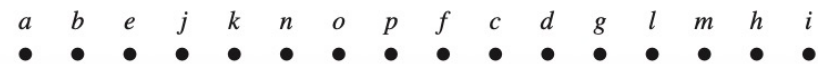
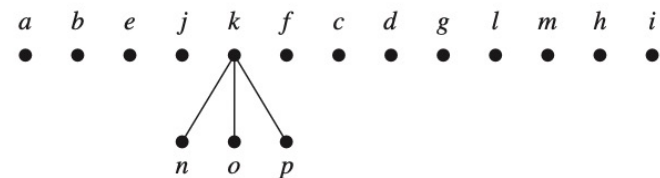
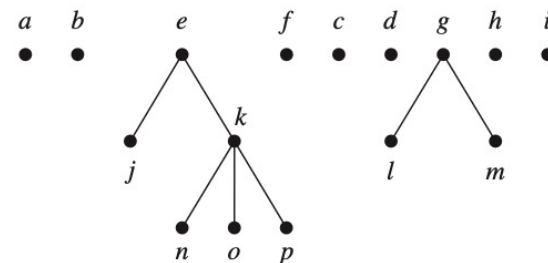
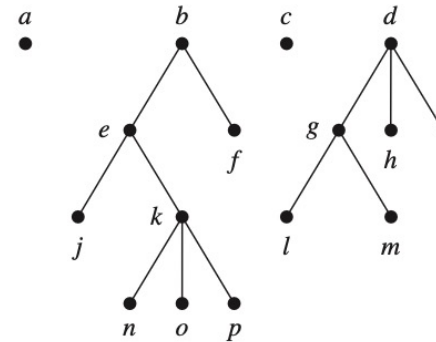
FIGURE 7 Postorder Traversal.

Duyệt cây

□ Ví dụ duyệt tiền tự

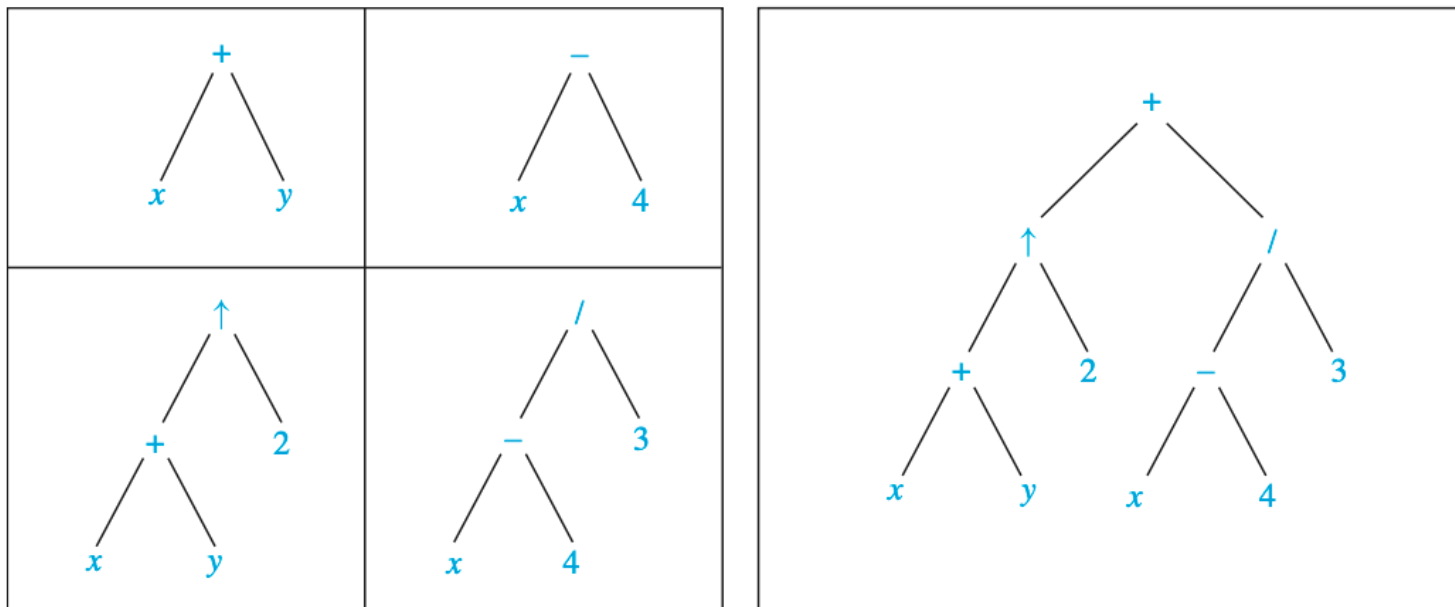


Preorder traversal: Visit root,
visit subtrees left to right



Biểu thức trung/tiền/hậu tố

□ Cây nhị phân biểu diễn biểu thức $((x + y) \uparrow 2) + ((x - 4)/3)$



□ Biểu thức tiền tố: Biểu thức/ký pháp Ba Lan

□ Biểu thức hậu tố: Biểu thức Ba Lan ngược

Biểu thức trung/tiền/hậu tố

❑ Tìm biểu thức Ba Lan của $((x + y) \uparrow 2) + ((x - 4)/3)$

❑ Tính giá trị biểu thức

$$+ \quad - \quad * \quad 2 \quad 3 \quad 5 \quad / \quad \underbrace{\uparrow \quad 2 \quad 3}_{} \quad 4$$

$2 \uparrow 3 = 8$

$$+ \quad - \quad * \quad 2 \quad 3 \quad 5 \quad / \quad \underbrace{8 \quad 4}_{} \quad$$

$8 / 4 = 2$

$$+ \quad - \quad \underbrace{* \quad 2 \quad 3}_{} \quad 5 \quad 2$$

$2 * 3 = 6$

$$+ \quad \underbrace{- \quad 6 \quad 5}_{} \quad 2$$

$6 - 5 = 1$

$$\underbrace{+ \quad 1 \quad 2}_{} \quad$$

$1 + 2 = 3$

Value of expression: 3

$$\underbrace{7 \quad 2 \quad 3 \quad *}_{} \quad - \quad 4 \quad \uparrow \quad 9 \quad 3 \quad / \quad +$$

$2 * 3 = 6$

$$\underbrace{7 \quad 6 \quad -}_{} \quad 4 \quad \uparrow \quad 9 \quad 3 \quad / \quad +$$

$7 - 6 = 1$

$$\underbrace{1 \quad 4 \quad \uparrow}_{} \quad 9 \quad 3 \quad / \quad +$$

$1^4 = 1$

$$1 \quad \underbrace{9 \quad 3 \quad /}_{} \quad +$$

$9 / 3 = 3$

$$\underbrace{1 \quad 3 \quad +}_{} \quad$$

$1 + 3 = 4$

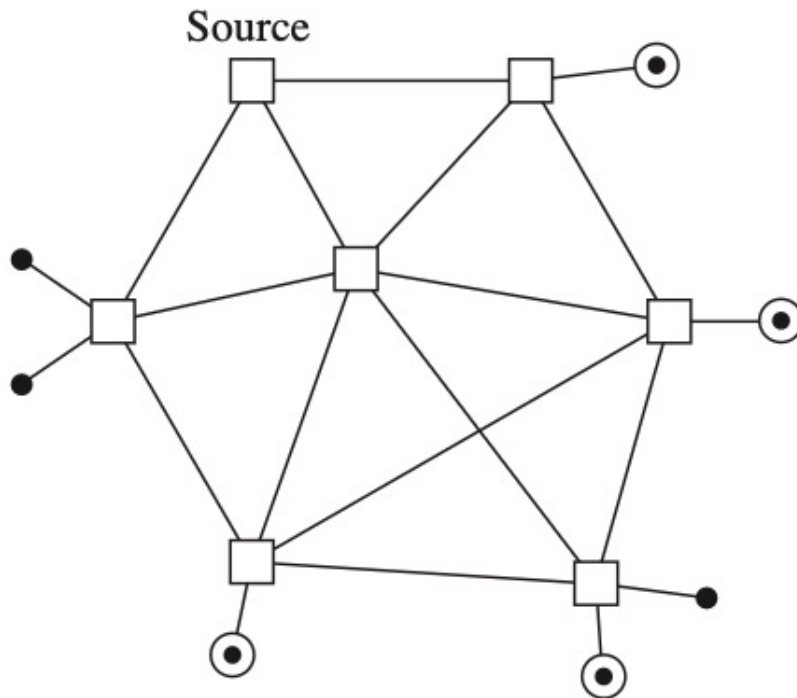
Value of expression: 4

Cây khung (spanning tree)

- Cây khung (bao trùm) của một đơn đồ thị G là một đồ thị con của G , chứa tất cả các đỉnh của G .
Một đơn đồ thị có cây bao trùm thì liên thông.
Một đơn đồ liên thông thì có cây bao trùm.
- **Định lý:** Một đơn đồ thị liên thông nếu và chỉ nếu nó có cây bao trùm.

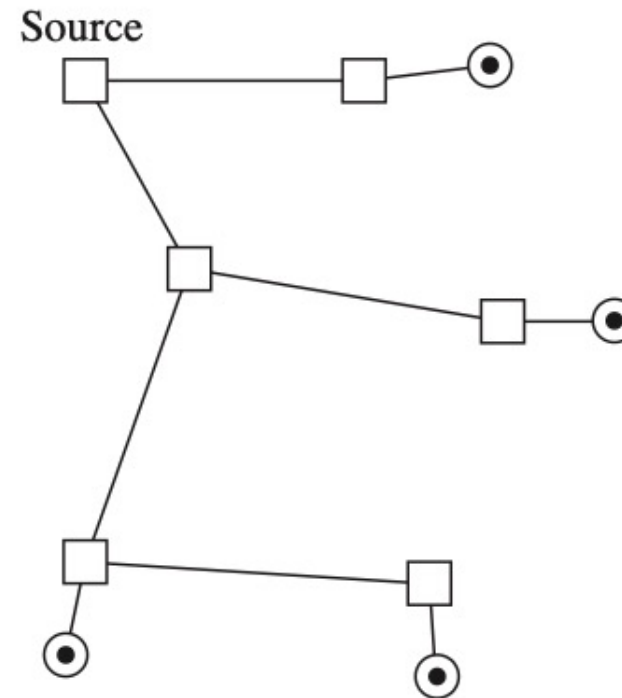
Cây khung (spanning tree)

Mạng IP



(a)

Cây khung đa trạm nhận



(b)

- Bộ định tuyến
- Mạng con
- ⊙ Mạng con có một trạm nhận

Cây khung

- ❑ Có thể loại bỏ các cạnh khỏi các chu trình đơn để tìm cây bao trùm trong một đồ thị

Không hiệu quả vì nó yêu cầu phải xác định được các chu trình đơn này.

- ❑ Thay vào đó, tìm bằng cách thêm các cạnh sử dụng các thuật toán duyệt đồ thị

- ❑ Theo chiều sâu (Depth-First Search)

- ❑ Theo chiều rộng (Breadth-First Search).

Tìm theo chiều sâu

Procedure DFS(G : đồ thị liên thông với các đỉnh $v_i, i = 1, \dots, n$)

$T :=$ cây có duy nhất một đỉnh v_1

visit(v_1)

Procedure visit(v : đỉnh của G)

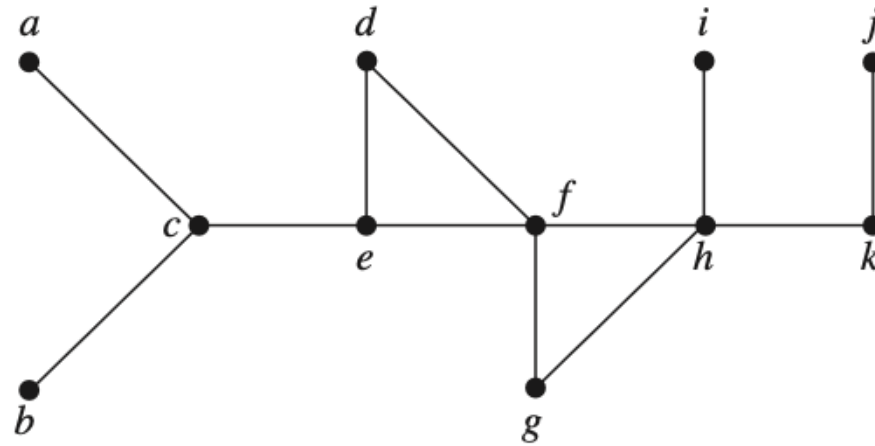
for mỗi đỉnh w kề với v và chưa nằm trong T

Thêm đỉnh w và cạnh $\{v, w\}$ vào T

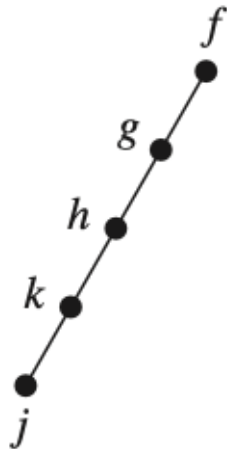
visit(w)

Thủ tục DFS xây dựng cây bao trùm với độ phức tạp là $O(e)$ hay $O(n^2)$ trong đó e và n lần lượt là số cạnh và số đỉnh của G

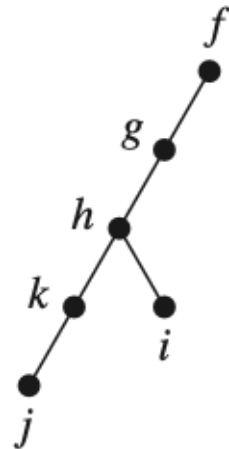
Tìm theo chiều sâu



f

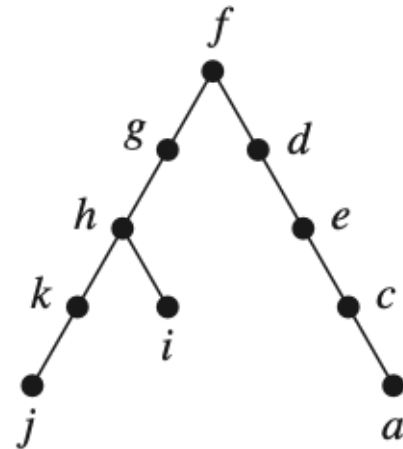


(a)

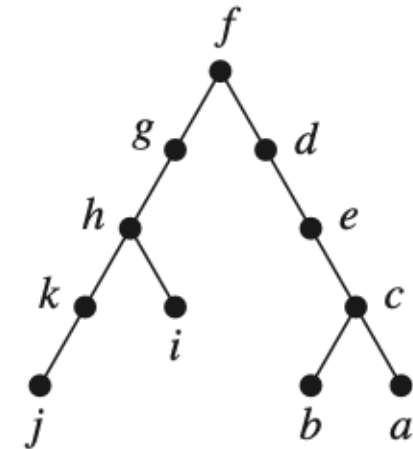


(b)

(c)



(d)



(e)

Tìm theo chiều rộng

Procedure BFS(G : đồ thị liên thông với các đỉnh $v_i, i = 1, \dots, n$)

$T :=$ cây có duy nhất một đỉnh v_1

$L :=$ danh sách rỗng (danh sách các đỉnh chưa được thăm)

Thêm v_1 vào L

while L không rỗng

 Lấy đỉnh đầu tiên, v , ra khỏi L

for mỗi lân cận w của v

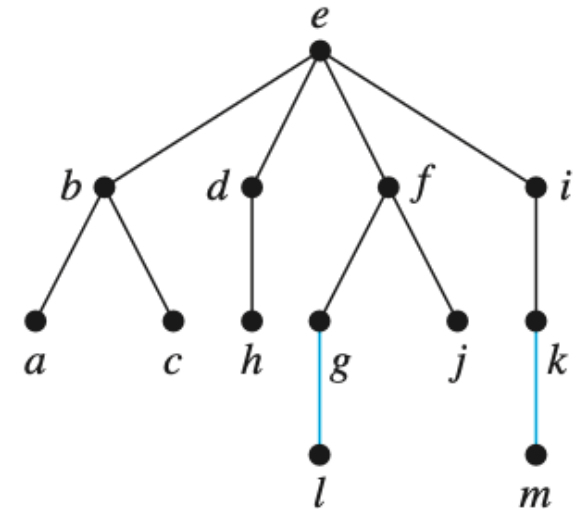
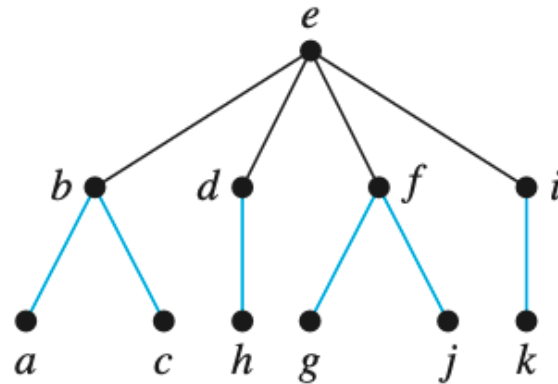
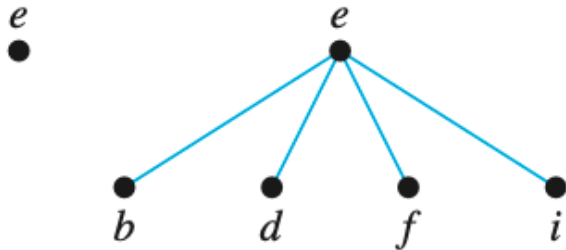
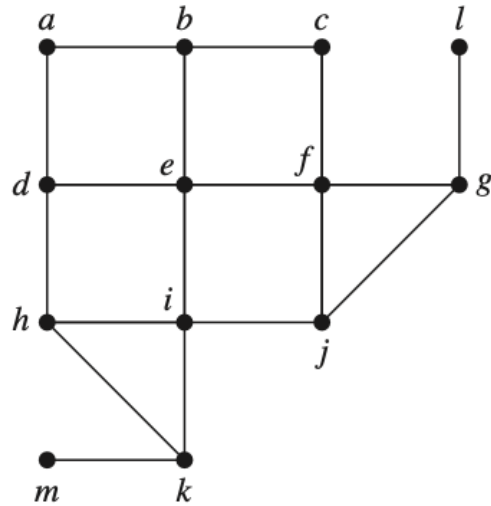
if w không nằm trong L và không trong T **then**

 Thêm w vào cuối L

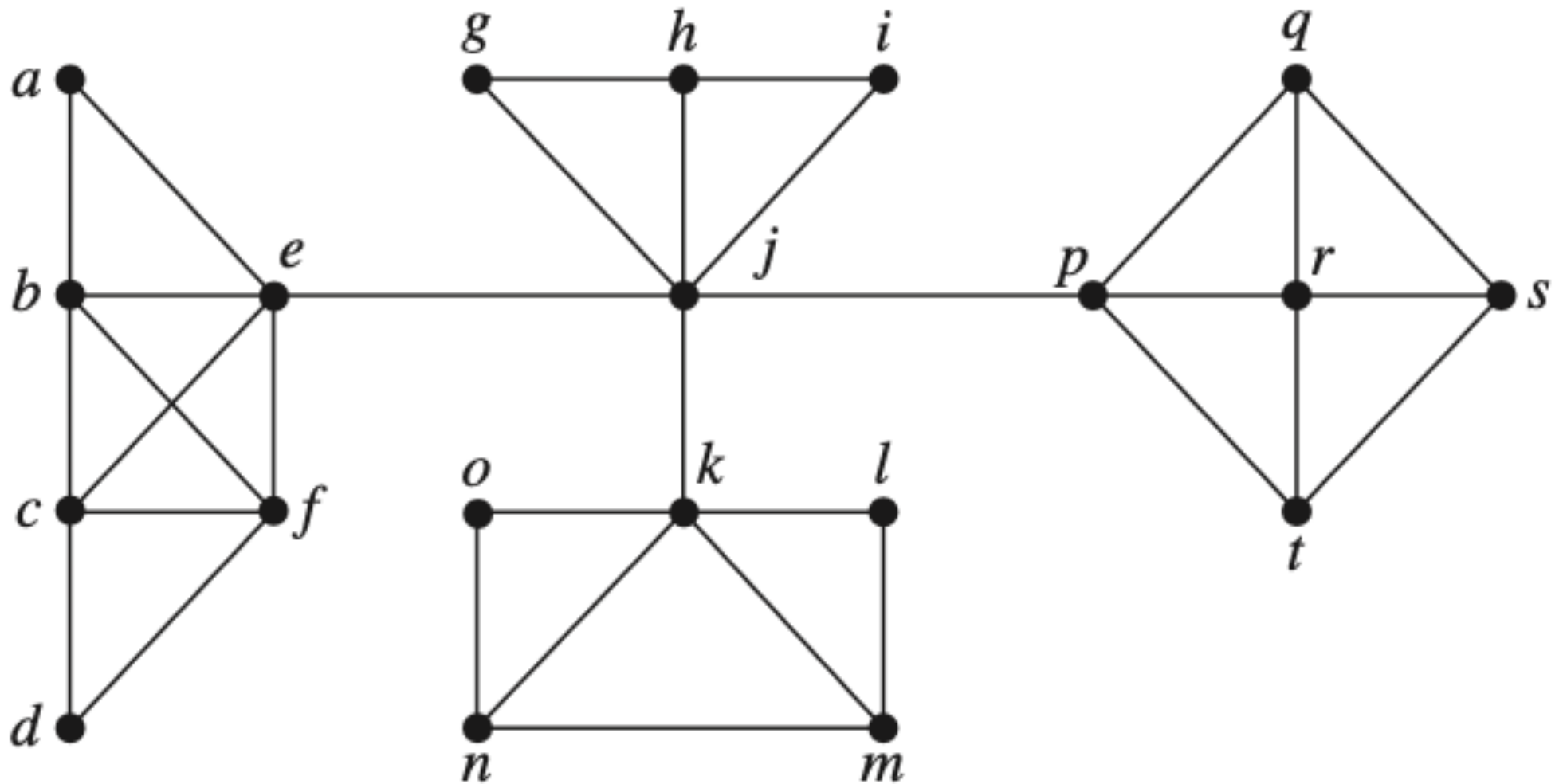
 Thêm w và cạnh $\{v, w\}$ vào T

Độ phức tạp của thuật toán BFS là $O(e)$ hay $O(n^2)$

Tìm theo chiều rộng



Thực hành duyệt cây tìm cây khung

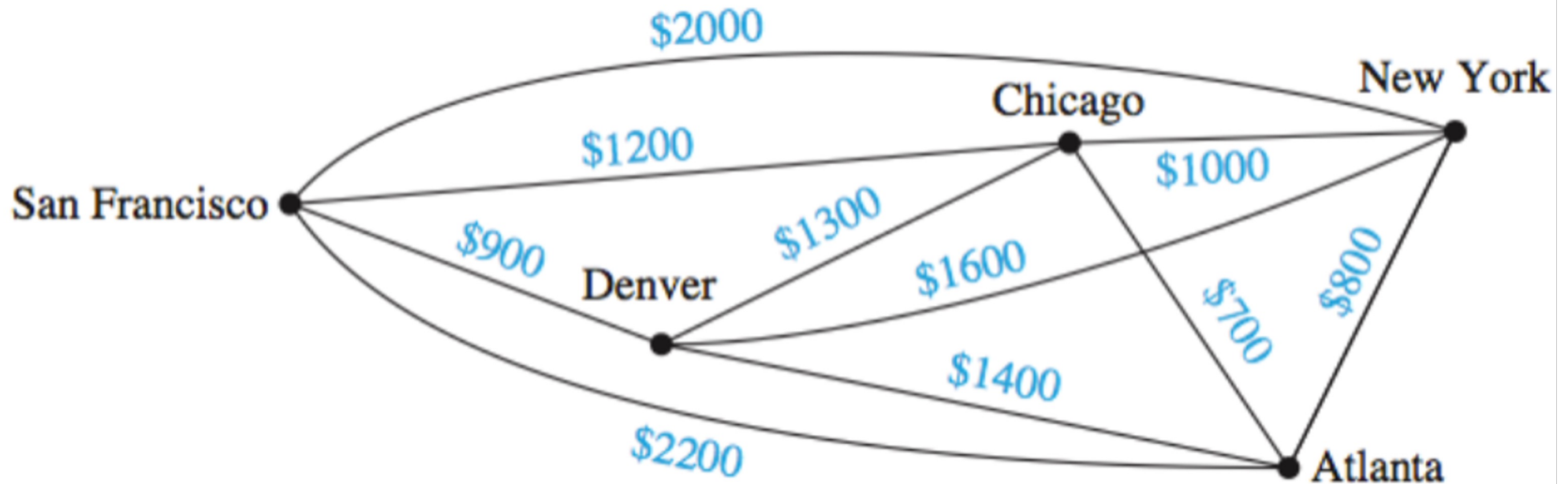


DFS trên đồ thị có hướng

- ❑ Ta có thể dễ dàng thay đổi 2 thuật toán DFS và BFS phù hợp với đồ thị có hướng.
- ❑ Đầu ra không nhất thiết là một cây bao trùm, mà đúng hơn là một rừng bao trùm

Nếu ở một giai đoạn của một trong hai thuật toán chúng ta thấy rằng không có cạnh nào tồn tại từ một đỉnh đã được thêm vào tới một đỉnh chưa được thêm vào, đỉnh tiếp theo được thêm bởi thuật toán trở thành gốc của một cây mới trong rừng bao trùm đó.

Cây bao trùm nhỏ nhất



Một đồ thị có trọng số thể hiện giá thuê bao hàng tháng cho các đường truyền trong một mạng máy tính

Tìm cây bao trùm mà tổng trọng số trên các cạnh của cây này là nhỏ nhất.

Cây bao trùm nhỏ nhất

□ Hai thuật toán tham lam đã được đề xuất để tìm cây bao trùm nhỏ nhất:

Thuật toán Prim

- Đề xuất bởi một nhà toán học người Séc vào năm 1930
- Phổ biến khi được tái nghiên cứu bởi Prim vào năm 1957

Thuật toán Kruskal

- Joseph Kruskal đề xuất vào năm 1956

Thuật toán Prim

Procedure Prim(G : đồ thị vô hướng liên thông có trọng số với n đỉnh)

$T :=$ một cạnh có trọng số nhỏ nhất

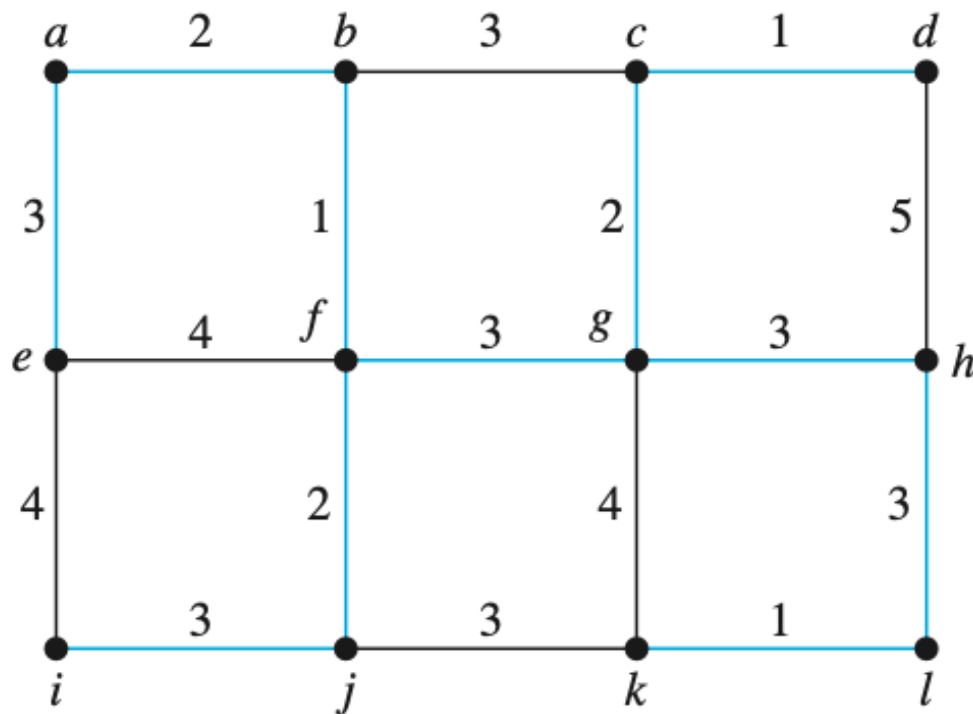
for $i := 1$ to $n-2$

$e :=$ một cạnh có trọng số nhỏ nhất có gắn với một đỉnh trong T và không tạo một chu trình trong T nếu được thêm vào T

$T := T$ với e được thêm vào

return T { T là cây khung/bao trùm nhỏ nhất của G }

Thuật toán Prim



(a)

Bước	Cạnh	Trọng số
1	$\{b, f\}$	1
2	$\{a, b\}$	2
3	$\{f, j\}$	2
4	$\{a, e\}$	3
5	$\{i, j\}$	3
6	$\{f, g\}$	3
7	$\{c, g\}$	2
8	$\{c, d\}$	1
9	$\{g, h\}$	3
10	$\{h, l\}$	3
11	$\{k, l\}$	1

Tổng: 24

(b)

Thuật toán Kruskal

Procedure Kruskal(G : đồ thị vô hướng liên thông có trọng số với n đỉnh)

$T :=$ đồ thị rỗng

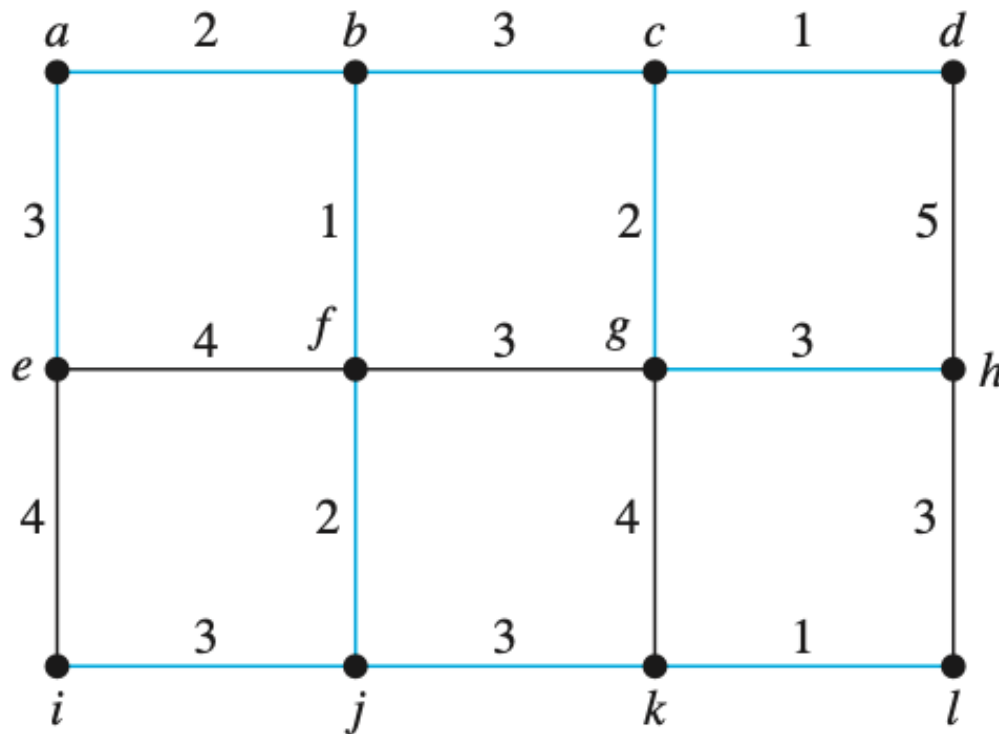
for $i := 1$ to $n-1$

$e :=$ một cạnh có trọng số nhỏ nhất không tạo một chu trình trong T nếu được thêm vào T

$T := T$ với e được thêm vào

return T { T là cây khung/bao trùm nhỏ nhất của G }

Thuật toán Kruskal



(a)

Bước	Cạnh	Trọng số
1	{c, d}	1
2	{k, l}	1
3	{b, f}	1
4	{c, g}	2
5	{a, b}	2
6	{f, j}	2
7	{b, c}	3
8	{j, k}	3
9	{g, h}	3
10	{i, j}	3
11	{a, e}	3
Tổng:		24

(b)

Thực hành Thuật toán Prim & Kruskal

